ESCUELA DE INGENIERÍA INFORMÁTICA



PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

TAREA 2: METAHEURÍSTICAS RECOCIDO SIMULADO PARA EL JOB SHOP SCHEDULING PROBLEM

AGUSTÍN GUZMÁN (21.572.150-7)

NICOLÁS LEIVA (21.855.076-2)

CRISTOBAL PINTO (21.994.622-8)

INVESTIGACIÓN DE OPERACIONES

PROFESORA: LESLIE PÉREZ CÁCERES

NOVIEMBRE 2026

RESUMEN

El presente informe describe la implementación y evaluación de una metaheurística de recocido simulado (*Simulated Annealing*) para resolver el problema de programación de talleres de trabajo (*Job Shop Scheduling Problem*, JSP). El objetivo es minimizar el tiempo total de término de todos los trabajos, conocido como *makespan*, respetando el orden tecnológico de las operaciones de cada trabajo y la capacidad de cada máquina. La solución se representa mediante una secuencia de identificadores de trabajo que siempre es factible y se decodifica con una regla de programación que asigna cada operación al primer instante disponible de su máquina y de su trabajo. El vecindario se genera intercambiando dos posiciones de la secuencia y la aceptación de soluciones peores sigue el criterio de Metropolis, con un esquema de enfriamiento geométrico.

La implementación se realizó en Rust y se evaluó sobre tres instancias de complejidad creciente: 6×6, 15×10 y 50×10. Para cada instancia se ejecutó el algoritmo diez veces con los mismos parámetros. La mejor solución encontrada alcanzó el óptimo en las tres instancias: 55, 1032 y 2794 unidades de tiempo, respectivamente. La instancia de tamaño medio presentó la mayor variabilidad, con una media de 1042,10 y una desviación estándar de 14,96, mientras que las instancias fácil y difícil alcanzaron el óptimo en las diez ejecuciones. Los resultados muestran que la metaheurística es simple, robusta y computacionalmente económica para las instancias estudiadas.

Palabras clave: Job Shop Scheduling, recocido simulado, metaheurística, makespan, optimización combinatoria.

ÍNDICE GENERAL

Resumen/Abstract **i**

Índice General **ii**

Lista de Figuras **iv**

Lista de Tablas **v**

Lista de Algoritmos **vi**

Lista de Símbolos **vii**

Glosario **viii**

1 Descripción general **1**

1.1 El problema de Job Shop Scheduling 1

1.2 Metaheurística seleccionada 1

1.2.1 Tipo de metaheurística 1

1.2.2 Ventajas y desventajas 2

1.2.3 Razón de la elección 2

2 Implementación **4**

2.1 Arquitectura del software 4

2.2 Representación de soluciones 5

2.3 Decodificación y evaluación 5

2.4 Generación de la solución inicial 6

2.5 Vecindario 6

2.6 Criterio de aceptación 6

2.7 Esquema de enfriamiento 6

2.8 Criterios de parada 6

2.9 Pseudocódigo 7

2.10 Diagrama de flujo 7
2.11 Parámetros y presupuesto computacional 8
2.12 Reproducibilidad 9

3 Mejor solución **10**

4 Resultados **11**

4.1 Configuración experimental 11
4.2 Resultados por ejecución 11
4.3 Resumen estadístico 11
4.4 Análisis de resultados 12
4.5 Gráfico de cajas 12

5 Conclusiones **14**

5.1 ¿Qué aprendió implementando la metaheurística? 14
5.2 ¿Cuál fue el desafío más grande de implementar la metaheurística? 14
5.3 ¿Tiene alguna idea para mejorar los resultados? 15
5.4 ¿Cree que este tipo de técnicas puede ser aplicada en la realidad chilena? 15
5.5 Conclusión general 15

Referencias **16**

A Instrucciones de ejecución **17**

A.1 Compilación y ejecución 17
A.2 Estructura del repositorio 17
A.3 Instancias utilizadas 18

LISTA DE FIGURAS

2.1 Diagrama de flujo del recocido simulado implementado. 8

4.1 Gráfico de cajas del makespan obtenido en diez ejecuciones por instancia. 13

LISTA DE TABLAS

2.1 Parámetros del recocido simulado utilizado en los experimentos. 8

3.1 Mejor solución encontrada para cada instancia. 10

4.1 Makespan obtenido en diez ejecuciones independientes por instancia. 11

4.2 Resumen estadístico de las diez ejecuciones por instancia. 12

A.1 Instancias utilizadas en la experimentación. 18

LISTA DE ALGORITMOS

2.1 Recocido simulado para el JSP 7

LISTA DE SÍMBOLOS

α	Factor de enfriamiento geométrico
Δ	Variación del makespan al pasar de una solución a un vecino
C_i	Instante de término del trabajo i
$C_{\max}$	Makespan o instante de término de todos los trabajos
J_i	Trabajo i, con $i = 1, \ldots, n$
m	Número de máquinas de la instancia
m_{ij}	Máquina requerida por la operación O_{ij}
n	Número de trabajos de la instancia
O_{ij}	Operación j-ésima del trabajo i
p_{ij}	Duración de la operación O_{ij}
s_{ij}	Instante de inicio de la operación O_{ij}
T	Temperatura actual del recocido simulado
T_0	Temperatura inicial
$T_{\min}$	Temperatura mínima de término
x	Secuencia de trabajos que representa una solución
x^*	Mejor secuencia encontrada durante la ejecución

GLOSARIO

CLI Command Line Interface. vii, 4, 5

CP-SAT Constraint Programming – SAT. vii, 10

JSP Job Shop Scheduling Problem. vii, 1, 7, 18

SA Simulated Annealing. vii

1. DESCRIPCIÓN GENERAL

1.1. El problema de Job Shop Scheduling

Formalmente, una instancia del Job Shop Scheduling Problem (JSP) está compuesta por un conjunto de n trabajos y un conjunto de m máquinas. Cada trabajo J_i, con $i = 1, \ldots, n$, debe ejecutar una secuencia ordenada de operaciones

$$O_{i1} \to O_{i2} \to \cdots \to O_{im_i},$$

donde m_i es el número de operaciones del trabajo i. Cada operación O_{ij} requiere la máquina m_{ij} durante un tiempo de procesamiento p_{ij} y no puede interrumpirse una vez iniciada. La secuencia de operaciones de cada trabajo representa su ruta tecnológica y debe respetarse estrictamente.

Sea s_{ij} el instante de inicio de la operación O_{ij}. Las restricciones de precedencia dentro de cada trabajo se expresan como

$$s_{i,j+1} \geq s_{ij} + p_{ij}, \qquad j = 1, \ldots, m_i - 1.$$

Además, dos operaciones que utilizan la misma máquina no pueden superponerse en el tiempo. Si O_{ij} y O_{kl} requieren la misma máquina, entonces debe cumplirse una de las dos condiciones siguientes:

$$s_{ij} + p_{ij} \leq s_{kl} \qquad \text{o} \qquad s_{kl} + p_{kl} \leq s_{ij}.$$

El instante de término del trabajo i es $C_i = s_{i,m_i} + p_{i,m_i}$, y el makespan se define como

$$C_{\max} = \max_{i=1,\ldots,n} C_i.$$

El objetivo del problema es encontrar una programación factible que minimice $C_{\max}$. En este informe se utilizan tres instancias de complejidad creciente: 6×6, 15×10 y 50×10, donde el primer valor corresponde al número de trabajos y el segundo al número de máquinas.

1.2. Metaheurística seleccionada

1.2.1. Tipo de metaheurística

El recocido simulado es una metaheurística de trayectoria, también denominada de solución única. A diferencia de las metaheurísticas poblacionales, como los algoritmos genéticos o la optimización por enjambre de partículas, el recocido simulado mantiene una sola solución actual y la modifica iterativamente mediante un operador de vecindario (Talbi, 2009). Su funcionamiento se inspira en el proceso físico de recocido de los metales: a altas temperaturas los átomos pueden reorganizarse y aceptar configuraciones de mayor energía; a medida que la temperatura disminuye, el sistema converge hacia configuraciones de menor energía (Kirkpatrick et al., 1983).

En el contexto de optimización, la "energía" corresponde al valor de la función objetivo y la "temperatura" controla la probabilidad de aceptar soluciones peores. Si x es la solución actual, x'

es un vecino y $\Delta = f(x') - f(x)$, el criterio de aceptación de Metropolis es

$$P(\text{aceptar } x') = \begin{cases} 1, & \text{si } \Delta < 0, \\ e^{-\Delta/T}, & \text{si } \Delta \geq 0, \end{cases}$$

donde $T > 0$ es la temperatura actual. Cuando T es alta, la probabilidad de aceptar un empeoramiento es considerable y el algoritmo explora ampliamente el espacio de búsqueda. Cuando T se aproxima a cero, la probabilidad de aceptar soluciones peores disminuye y el algoritmo se comporta como una búsqueda local descendente.

1.2.2. Ventajas y desventajas

Las principales ventajas del recocido simulado son:

- Es sencillo de implementar y requiere pocos parámetros: temperatura inicial, temperatura mínima, factor de enfriamiento y criterio de parada.
- Permite escapar de óptimos locales gracias a la aceptación probabilística de soluciones peores.
- Es un algoritmo *anytime*: en cualquier momento de la ejecución se dispone de una solución factible y de la mejor solución encontrada hasta ese instante.
- Se adapta con facilidad a problemas combinatorios en los que es posible definir un vecindario y una función objetivo.
- Tiene un bajo costo computacional por iteración cuando la evaluación del vecindario puede realizarse de forma eficiente.

Sus principales desventajas son:

- No garantiza encontrar la solución óptima ni permite certificar optimalidad.
- Su desempeño es sensible a la calibración de la temperatura inicial, la temperatura mínima y el factor de enfriamiento.
- Un enfriamiento demasiado rápido puede conducir a un óptimo local de baja calidad; un enfriamiento demasiado lento puede consumir un tiempo excesivo.
- Mantiene una sola solución, por lo que no aprovecha la diversidad de una población.
- La calidad de la solución depende del vecindario elegido y de la representación utilizada.

1.2.3. Razón de la elección

Se eligió el recocido simulado por tres razones principales. Primero, el problema estudiado es de naturaleza combinatoria y el recocido simulado ha demostrado ser eficaz en problemas de programación de la producción (Pinedo, 2016). Segundo, el algoritmo es fácil de explicar, implementar y reproducir, lo que facilita la entrega y la presentación del trabajo. Tercero, el grupo ya contaba con una implementación base en Rust, lo que permitió concentrar el esfuerzo en la representación de soluciones, la decodificación y la experimentación, en lugar de partir desde cero.

Además, el recocido simulado ofrece un buen equilibrio entre calidad y costo computacional para las tres instancias propuestas. Como se muestra en el Capítulo 4, con un presupuesto de aproximadamente 27.625 iteraciones por ejecución se alcanzó el óptimo en las tres instancias, lo que confirma que la elección fue adecuada para el alcance del trabajo.

2. IMPLEMENTACIÓN

2.1. Arquitectura del software

La implementación se desarrolló en Rust, edición 2024, y se organizó en tres módulos principales: `domain`, `solver` y `ui`. El módulo `domain` contiene las estructuras que representan la instancia y las soluciones: `Jsp`, `Job`, `Operation`, `Schedule` y `ScheduleItem`. El módulo `solver` define los *traits* `Solver` y `SolutionBuilder`, las condiciones de parada y las estrategias de resolución. El módulo `ui` implementa la interfaz de línea de comandos, la barra de progreso y la selección de estrategia.

```
src/
|-- domain/
|   |-- jsp.rs
|   |-- job.rs
|   |-- operation.rs
|   `-- solution/
|       |-- schedule.rs
|       `-- schedule_item.rs
|-- solver/
|   |-- solution/
|   |   |-- solution_builder.rs
|   |   |-- solver.rs
|   |   `-- strategies/
|   |       |-- simulated_annealing.rs
|   |       |-- local_search.rs
|   |       |-- random.rs
|   |       |-- swap.rs
|   |       `-- bruteforce.rs
|   `-- stop/
|       |-- cancel.rs
|       |-- timeout.rs
|       `-- stop_handle.rs
`-- ui/
    |-- cli.rs
    |-- progress.rs
    `-- solver_data.rs
```

La interfaz de línea de comandos (Command Line Interface (CLI)) se implementó con la biblioteca `clap`. El programa recibe la ruta de la instancia, un tiempo límite opcional y el subcomando que selecciona la estrategia. Para el recocido simulado, los parámetros se ingresan mediante opciones con valores por defecto:

```
./target/release/jsp -f data/facil.txt simulated-annealing
./target/release/jsp -f data/medio.txt simulated-annealing \
    -i 1000.0 -m 0.001 -a 0.9995
```

El binario `benchmark` ejecuta automáticamente diez repeticiones por instancia, extrae el makespan desde la salida de la CLI y calcula la media y la desviación estándar muestral. De esta forma, los resultados del informe pueden reproducirse con un solo comando.

El código fue desarrollado íntegramente por los integrantes del grupo. La única referencia externa es la descripción original del recocido simulado (Kirkpatrick et al., 1983), citada en el Capítulo 1.

2.2. Representación de soluciones

Una solución se representa mediante un vector de identificadores de trabajos, $x = (x_1, \ldots, x_N)$, donde $N = \sum_{i=1}^{n} m_i$ es el número total de operaciones. Cada trabajo i aparece exactamente m_i veces en el vector. Por ejemplo, si una instancia tiene tres trabajos con tres operaciones cada uno, una solución puede ser

$$x = (2, 1, 3, 1, 2, 3, 1, 2, 3).$$

La k-ésima aparición del trabajo i en la secuencia indica que debe ejecutarse la k-ésima operación de ese trabajo. Esta representación tiene dos ventajas importantes: siempre conserva la multiplicidad de operaciones y permite decodificar una solución factible sin necesidad de aplicar reparaciones.

2.3. Decodificación y evaluación

La función de decodificación recorre la secuencia de izquierda a derecha y mantiene dos arreglos: el instante de término de cada trabajo y el instante de liberación de cada máquina. Para cada identificador de trabajo i se toma su siguiente operación pendiente O_{ij}, se calcula

$$s_{ij} = \max\left(\text{término del trabajo } i,\ \text{liberación de la máquina } m_{ij}\right),$$

y se actualizan ambos instantes con $s_{ij} + p_{ij}$. Al finalizar el recorrido, el makespan es el máximo de los instantes de término de los trabajos. El costo de una evaluación es $O(N)$, por lo que el algoritmo puede realizar decenas de miles de evaluaciones en tiempos muy reducidos.

El decodificador se implementó en el rasgo `SolutionBuilder`, con métodos separados para evaluar el makespan y para construir la programación completa. El primero retorna únicamente el valor de la función objetivo; el segundo construye la lista de `ScheduleItem` con job, operación, máquina, inicio, término y duración, necesaria para mostrar la programación final.

Como validación del decodificador, se ejecutó la estrategia de fuerza bruta sobre la instancia mínima de 3×3. El resultado fue un makespan de 30, que coincide con el óptimo de esa instancia y con el ejemplo documentado en el repositorio. Esta prueba confirma que la lógica de precedencias y de capacidad de las máquinas es correcta.

2.4. Generación de la solución inicial

La solución inicial se genera mediante una permutación aleatoria del vector base que contiene cada trabajo repetido según su número de operaciones. Para ello se utiliza el generador de números aleatorios de la biblioteca `rand`. Cada ejecución parte de una solución distinta, lo que permite medir la variabilidad del algoritmo.

2.5. Vecindario

El operador de vecindario selecciona aleatoriamente dos posiciones i y j de la secuencia, con la condición de que $x_i \neq x_j$, y las intercambia. La condición evita intercambios que no modifican la solución. Este movimiento se conoce como *swap* y preserva la multiplicidad de cada trabajo, por lo que el vecino siempre es una solución factible. El vecindario completo de una solución está formado por todos los intercambios posibles entre posiciones que contienen trabajos distintos.

2.6. Criterio de aceptación

Sea x la solución actual, x' un vecino generado por el operador de swap y $\Delta = f(x') - f(x)$. Si $\Delta < 0$, el vecino mejora la solución y se acepta siempre. Si $\Delta \geq 0$, se acepta con probabilidad $e^{-\Delta/T}$, donde T es la temperatura actual. Este criterio permite que el algoritmo acepte empeoramientos al inicio de la búsqueda y que los rechace con mayor frecuencia a medida que la temperatura disminuye. La mejor solución encontrada se almacena por separado, de modo que un empeoramiento nunca afecta el resultado final.

2.7. Esquema de enfriamiento

Se utilizó un enfriamiento geométrico:

$$T_{k+1} = \alpha T_k,$$

con $T_0 = 1000$, $T_{\min} = 0.001$ y $\alpha = 0.9995$. El número de iteraciones hasta alcanzar la temperatura mínima es

$$k_{\max} = \left\lceil \frac{\ln(T_{\min}/T_0)}{\ln \alpha} \right\rceil = \left\lceil \frac{\ln(10^{-6})}{\ln(0.9995)} \right\rceil \approx 27.625.$$

En cada iteración se evalúa un vecino, por lo que una ejecución realiza aproximadamente 27.626 evaluaciones de la función objetivo, incluida la evaluación de la solución inicial. Este presupuesto es fijo e independiente de la instancia, lo que permite comparar las tres instancias bajo las mismas condiciones.

2.8. Criterios de parada

El algoritmo se detiene cuando la temperatura alcanza $T_{\min}$. Adicionalmente, la implementación admite dos criterios de parada opcionales: un tiempo límite especificado con la opción `-timeout` y la interrupción manual mediante `Ctrl+C`. Las condiciones de parada se gestionan a través de `StopHandle`, que evalúa todas las condiciones activas y detiene la búsqueda cuando al menos

una de ellas se cumple. Cuando esto ocurre, se retorna la mejor solución encontrada hasta ese momento.

2.9. Pseudocódigo

El Algoritmo 2.1 resume el funcionamiento del recocido simulado implementado. La decodificación de una secuencia se denota mediante la función DECODIFICAR, que retorna el makespan correspondiente.

Algoritmo 2.1 Recocido simulado para el JSP

Require: Instancia del JSP, T_0, $T_{\min}$, α
Ensure: Mejor secuencia encontrada x^*

Generar una permutación aleatoria x del vector base de trabajos
$f(x) \leftarrow \text{DECODIFICAR}(x)$
$x^* \leftarrow x$, $f^* \leftarrow f(x)$
$T \leftarrow T_0$
while $T > T_{\min}$ y no se cumpla un criterio de parada **do**
    Elegir dos posiciones aleatorias i, j con $x_i \neq x_j$
    $x' \leftarrow x$ con las posiciones i y j intercambiadas
    $f(x') \leftarrow \text{DECODIFICAR}(x')$
    $\Delta \leftarrow f(x') - f(x)$
    if $\Delta < 0$ **o** aleatorio$(0, 1) < e^{-\Delta/T}$ **then**
        $x \leftarrow x'$
        if $f(x) < f^*$ **then**
            $x^* \leftarrow x$
            $f^* \leftarrow f(x)$
        end if
    end if
    $T \leftarrow \alpha T$
end while
return x^*

2.10. Diagrama de flujo

La Figura 2.1 reserva el espacio para el diagrama de flujo del algoritmo. El diagrama se incorporará en la versión final del informe.

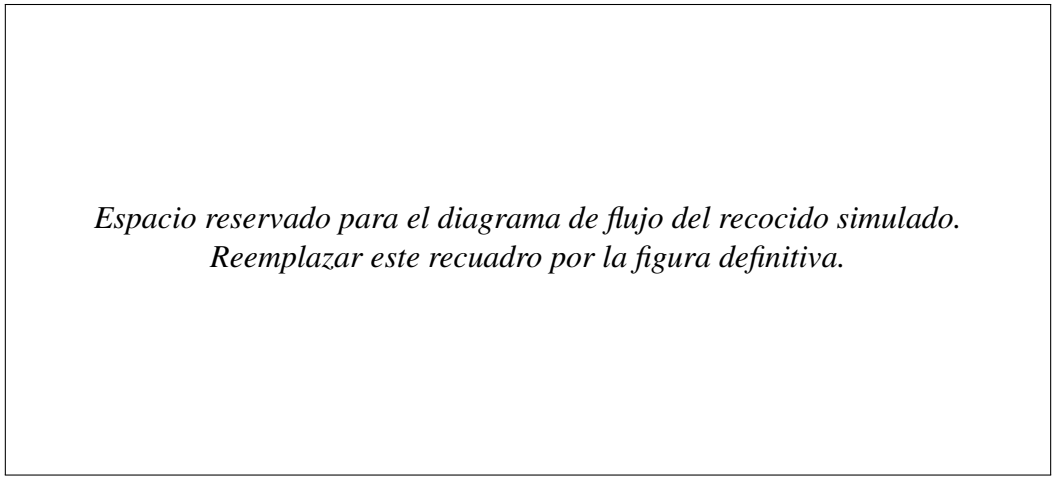


Figura 2.1: Diagrama de flujo del recocido simulado implementado.

2.11. Parámetros y presupuesto computacional

La Tabla 2.1 resume los parámetros utilizados en todos los experimentos. No se realizó ajuste específico por instancia: las tres instancias se resolvieron con el mismo conjunto de parámetros, lo que permite comparar el comportamiento del algoritmo bajo condiciones homogéneas.

Tabla 2.1: Parámetros del recocido simulado utilizado en los experimentos.

Parámetro	Símbolo	Valor
Temperatura inicial	T_0	1000
Temperatura mínima	$T_{\min}$	0,001
Factor de enfriamiento	α	0,9995
Iteraciones por ejecución	$k_{\max}$	27.625
Evaluaciones por ejecución	–	27.626
Ejecuciones por instancia	–	10
Tiempo límite	–	Desactivado
Semilla	–	Aleatoria del sistema
Vecindario	–	Swap de dos posiciones
Criterio de aceptación	–	Metropolis

El presupuesto total de la experimentación fue de $3 \times 10 \times 27.626 = 828.780$ evaluaciones de la función objetivo. Los tiempos de ejecución medidos en el equipo de pruebas fueron aproximadamente 0,007 s para la instancia fácil, 0,011 s para la instancia media y 0,028 s para la instancia difícil, incluyendo el tiempo de arranque del proceso. Estos valores corresponden a un procesador Intel Core i3-1215U con 8 GB de memoria RAM y Rust 1.96.0 en Linux.

2.12. Reproducibilidad

El repositorio incluye un archivo `README.txt` con las instrucciones de compilación y ejecución, además de la identificación de los integrantes del grupo. Para compilar y ejecutar el algoritmo se utilizan los siguientes comandos:

```
cargo build --release
./target/release/jsp -f data/facil.txt simulated-annealing
./target/release/jsp -f data/medio.txt simulated-annealing
./target/release/jsp -f data/dificil.txt simulated-annealing
```

Para reproducir la tabla completa de diez ejecuciones por instancia se ejecuta:

```
./target/release/benchmark
```

El binario `benchmark` recorre las instancias `facil.txt`, `medio.txt` y `dificil.txt`, ejecuta el recocido simulado diez veces por instancia y reporta el mejor valor, el peor valor, la media y la desviación estándar.

3. MEJOR SOLUCIÓN

Para cada instancia se registró la mejor solución encontrada entre las diez ejecuciones independientes. La Tabla 3.1 presenta el makespan de la mejor solución, el valor óptimo de referencia y la brecha correspondiente. Los valores óptimos fueron verificados con un modelo de programación de restricciones resuelto con Constraint Programming – SAT (CP-SAT) de OR-Tools (Google, 2026), utilizado únicamente como referencia de evaluación y no como parte de la metaheurística implementada.

Tabla 3.1: Mejor solución encontrada para cada instancia.

Instancia	Tamaño	Mejor $C_{\max}$	Óptimo	Brecha	Ejecuciones óptimas
Fácil	6×6	55	55	0,00 %	10/10
Medio	15×10	1032	1032	0,00 %	6/10
Difícil	50×10	2794	2794	0,00 %	10/10

La mejor solución de la instancia fácil se obtuvo en las diez ejecuciones, con un makespan de 55. En la instancia media, el óptimo de 1032 se alcanzó en seis de las diez ejecuciones; las cuatro restantes quedaron entre 1038 y 1077. En la instancia difícil, las diez ejecuciones alcanzaron el valor 2794, que coincide con la cota inferior dada por la carga total de la máquina más congestionada.

La brecha se calculó como

$$\text{brecha} = \frac{C_{\max}^{\text{mejor}} - C_{\max}^{\text{óptimo}}}{C_{\max}^{\text{óptimo}}} \times 100\,\%.$$

En las tres instancias la mejor solución encontrada es óptima, por lo que la brecha es cero. La diferencia entre ejecuciones se analiza con mayor detalle en el Capítulo 4.

La programación completa de la mejor solución puede reconstruirse ejecutando el binario sobre la instancia correspondiente. Por ejemplo, para la instancia fácil:

```
./target/release/jsp -f data/facil.txt simulated-annealing
```

La salida incluye el makespan y una tabla con el trabajo, la operación, la máquina, el instante de inicio, el instante de término y la duración de cada operación.

4. RESULTADOS

4.1. Configuración experimental

Los experimentos se realizaron con el binario compilado en modo `release`. Para cada una de las tres instancias se ejecutó el recocido simulado diez veces de forma independiente, utilizando los parámetros de la Tabla 2.1 y sin tiempo límite. Cada ejecución parte de una permutación inicial aleatoria distinta, por lo que los resultados reflejan la variabilidad propia del algoritmo.

El equipo de pruebas fue un computador con procesador Intel Core i3-1215U, 8 GB de memoria RAM y sistema operativo Linux. La implementación utilizó Rust 1.96.0 y las bibliotecas `clap`, `rand`, `indicatif`, `ctrlc` y `thiserror`. El binario `benchmark` automatizó las treinta ejecuciones y el cálculo de los estadísticos.

4.2. Resultados por ejecución

La Tabla 4.1 presenta el makespan obtenido en cada una de las diez ejecuciones para las tres instancias, junto con la media y la desviación estándar muestral.

Tabla 4.1: Makespan obtenido en diez ejecuciones independientes por instancia.

Ejecución	Fácil (6×6)	Medio (15×10)	Difícil (50×10)
1	55	1077	2794
2	55	1053	2794
3	55	1054	2794
4	55	1039	2794
5	55	1032	2794
6	55	1032	2794
7	55	1032	2794
8	55	1032	2794
9	55	1038	2794
10	55	1032	2794
Media	55,00	1042,10	2794,00
Desviación estándar	0,00	14,96	0,00

4.3. Resumen estadístico

La Tabla 4.2 resume los principales estadísticos por instancia y los compara con el óptimo. La brecha media se calcula como la diferencia porcentual entre la media de las diez ejecuciones y el óptimo.

Tabla 4.2: Resumen estadístico de las diez ejecuciones por instancia.

Instancia	Óptimo	Mejor	Peor	Media	Desv. est.	Brecha media
Fácil	55	55	55	55,00	0,00	0,00 %
Medio	1032	1032	1077	1042,10	14,96	0,98 %
Difícil	2794	2794	2794	2794,00	0,00	0,00 %

4.4. Análisis de resultados

En la instancia fácil, las diez ejecuciones alcanzaron el mismo makespan de 55, que corresponde al óptimo. La desviación estándar nula indica que el algoritmo fue completamente estable en esta instancia. El tamaño reducido del problema (6×6, 36 operaciones) permite que el presupuesto de 27.625 iteraciones explore el espacio de búsqueda con suficiente profundidad, incluso partiendo de soluciones iniciales aleatorias distintas.

En la instancia media, seis de las diez ejecuciones alcanzaron el óptimo de 1032, mientras que las cuatro restantes obtuvieron valores entre 1038 y 1077. La media fue 1042,10 y la desviación estándar 14,96, lo que representa una brecha media de solo 0,98 % respecto del óptimo. La mayor variabilidad se explica por el tamaño del problema (15×10, 150 operaciones) y por la naturaleza estocástica del recocido simulado: la calidad del óptimo local alcanzado depende de la trayectoria seguida y de la solución inicial. Aun así, el peor resultado (1077) se encuentra a solo 4,36 % del óptimo, lo que muestra que el algoritmo mantiene una calidad aceptable incluso en sus ejecuciones menos favorables.

En la instancia difícil, las diez ejecuciones alcanzaron el valor 2794, que coincide con la cota inferior determinada por la máquina más cargada. La desviación estándar nula y la coincidencia con el óptimo indican que, para esta instancia, la estructura del problema guía al algoritmo hacia una solución de alta calidad de forma consistente. El mayor número de operaciones (50×10, 500 operaciones) no impidió alcanzar el óptimo, aunque el costo por evaluación es mayor que en las otras instancias.

En términos generales, el recocido simulado mostró un comportamiento robusto: con un único conjunto de parámetros y sin ajuste por instancia, alcanzó el óptimo en las tres instancias y mantuvo una brecha media inferior al 1 % en el caso más variable. El presupuesto de aproximadamente 27.625 iteraciones por ejecución resultó suficiente para las instancias estudiadas y el costo computacional fue bajo: menos de 0,03 s por ejecución en el equipo de pruebas.

4.5. Gráfico de cajas

La Figura 4.1 reserva el espacio para el gráfico de cajas de los makespan obtenidos en las diez ejecuciones de cada instancia. El gráfico permitirá visualizar la dispersión, la mediana y los valores extremos de cada conjunto de resultados.

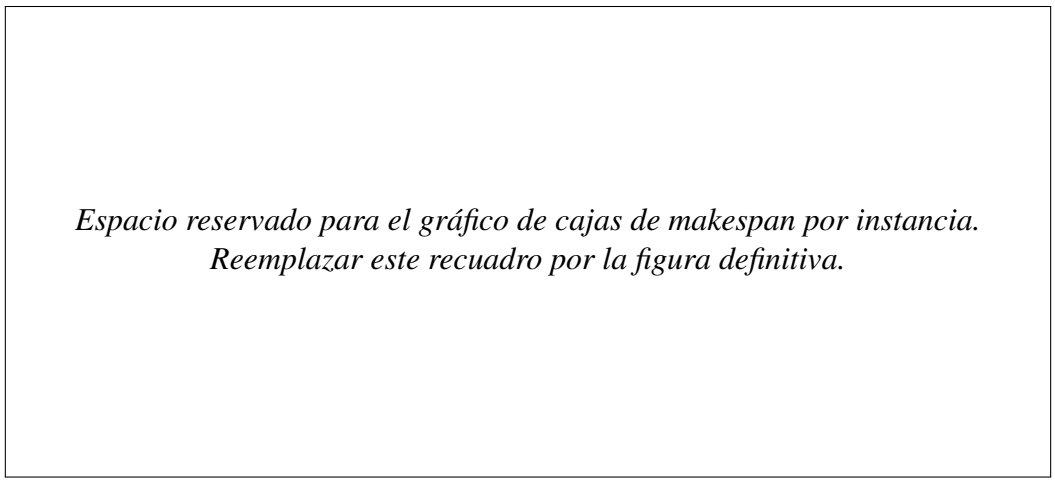


Figura 4.1: Gráfico de cajas del makespan obtenido en diez ejecuciones por instancia.

La instancia fácil y la instancia difícil presentan una dispersión nula, por lo que sus cajas se reducen a una línea en el valor óptimo. La instancia media concentra la mayor dispersión, con seis ejecuciones en 1032 y cuatro valores superiores. Este comportamiento es consistente con el análisis de la sección anterior y sugiere que el algoritmo es más sensible a la trayectoria en problemas de tamaño intermedio.

5. CONCLUSIONES

En este trabajo se implementó y evaluó una metaheurística de recocido simulado para resolver el problema de Job Shop Scheduling. El algoritmo se desarrolló íntegramente en Rust, con una representación de soluciones basada en secuencias de trabajos y un decodificador que garantiza el cumplimiento de las precedencias y de la capacidad de las máquinas. Se evaluaron tres instancias de complejidad creciente y se realizaron diez ejecuciones independientes por instancia.

Los resultados muestran que la mejor solución encontrada alcanzó el óptimo en las tres instancias: 55 para la instancia fácil, 1032 para la instancia media y 2794 para la instancia difícil. La instancia media presentó la mayor variabilidad, con una media de 1042,10 y una desviación estándar de 14,96, mientras que las instancias fácil y difícil alcanzaron el óptimo en las diez ejecuciones. La brecha media en el caso más variable fue inferior al 1 %, lo que confirma la calidad del algoritmo con un presupuesto computacional reducido.

5.1. ¿Qué aprendió implementando la metaheurística?

La implementación permitió comprender que el desempeño de una metaheurística no depende únicamente del algoritmo, sino también de la representación de las soluciones, de la función de decodificación y del vecindario. Una representación adecuada puede garantizar la factibilidad de todas las soluciones y simplificar enormemente la búsqueda. También se aprendió que el esquema de enfriamiento y el criterio de aceptación son fundamentales para equilibrar exploración y explotación, y que la experimentación sistemática con varias ejecuciones es necesaria para medir la variabilidad del método.

Desde el punto de vista de la ingeniería de software, el trabajo permitió aplicar conceptos de diseño modular, rasgos y separación de responsabilidades en Rust, además de automatizar la experimentación mediante un binario de benchmark.

5.2. ¿Cuál fue el desafío más grande de implementar la metaheurística?

El desafío más grande fue diseñar una representación que preservara la factibilidad de las soluciones y, al mismo tiempo, permitiera evaluar el makespan de forma eficiente. La secuencia de trabajos con repeticiones debe decodificarse respetando tanto el orden de las operaciones de cada trabajo como la disponibilidad de las máquinas; cualquier error en esta lógica produce soluciones infactibles o valores de makespan incorrectos. La validación con la instancia mínima y con la estrategia de fuerza bruta fue clave para detectar y corregir este tipo de errores.

Otro desafío importante fue la calibración de los parámetros. Un enfriamiento demasiado rápido reduce la exploración y puede estancar el algoritmo en un óptimo local; un enfriamiento demasiado lento incrementa innecesariamente el tiempo de ejecución. Encontrar un equilibrio adecuado requirió probar distintas configuraciones y analizar su efecto sobre la calidad de las soluciones.

5.3. ¿Tiene alguna idea para mejorar los resultados?

Existen varias líneas de mejora. En primer lugar, se podría utilizar un vecindario más informado, por ejemplo, intercambios basados en la ruta crítica o movimientos de inserción en lugar de swaps aleatorios. En segundo lugar, se podría implementar un esquema de enfriamiento adaptativo o con recalentamientos periódicos para escapar de óptimos locales profundos. En tercer lugar, se podría combinar el recocido simulado con una búsqueda local intensiva al final de la ejecución, o ejecutar varias réplicas en paralelo y quedarse con la mejor solución. Finalmente, se podrían ajustar los parámetros por instancia mediante un estudio experimental más amplio o utilizar un enfoque de *tuning* automático.

Otra mejora posible es incorporar información específica del problema, como cotas inferiores por carga de máquina, para guiar la búsqueda o para detener el algoritmo cuando se alcanza una solución que coincide con la cota. Esto permitiría ahorrar evaluaciones en instancias donde el óptimo está determinado por la máquina más congestionada.

5.4. ¿Cree que este tipo de técnicas puede ser aplicada en la realidad chilena?

Sí. Las metaheurísticas como el recocido simulado son especialmente útiles en contextos donde los métodos exactos no son viables por el tamaño o la complejidad de los problemas. En Chile, este tipo de técnicas puede aplicarse en la planificación de la producción de pequeñas y medianas empresas manufactureras, en la programación de talleres de mantenimiento, en la gestión de turnos y en la logística de distribución. La principal barrera no es el algoritmo, sino la disponibilidad de datos confiables y la capacidad de integrar la solución con los sistemas de información existentes.

Además, el bajo costo computacional del recocido simulado permite ejecutarlo en computadores personales o en servidores modestos, sin necesidad de infraestructura especializada. Esto lo convierte en una alternativa accesible para organizaciones que no pueden invertir en soluciones comerciales de optimización. Con una adecuada capacitación y un proceso de validación con datos reales, este tipo de técnicas podría aportar mejoras concretas en eficiencia y en tiempos de respuesta.

5.5. Conclusión general

El recocido simulado demostró ser una metaheurística adecuada para el problema estudiado: simple de implementar, fácil de parametrizar y capaz de alcanzar el óptimo en las tres instancias con un presupuesto computacional reducido. La combinación de una representación factible, un decodificador eficiente y un criterio de aceptación probabilístico permitió obtener resultados de alta calidad y una variabilidad controlada. El trabajo deja abiertas varias líneas de mejora que podrían aumentar la robustez del algoritmo en instancias de mayor tamaño o en problemas con restricciones adicionales.

REFERENCIAS

Google. (2026). OR-Tools CP-SAT solver [Accedido el 30 de octubre de 2026]. https://developers.google.com/optimization

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, *220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Pinedo, M. L. (2016). *Scheduling: Theory, algorithms, and systems* (5th ed.). Springer. https://doi.org/10.1007/978-3-319-26580-3

Talbi, E.-G. (2009). *Metaheuristics: From design to implementation*. John Wiley & Sons.

A. INSTRUCCIONES DE EJECUCIÓN

A.1. Compilación y ejecución

El código se compila con Cargo, el sistema de construcción de Rust. Desde la raíz del repositorio se ejecuta:

```
cargo build --release
```

El binario principal se encuentra en `target/release/jsp` y recibe la ruta de la instancia, un tiempo límite opcional y el subcomando de la estrategia. Para el recocido simulado con los parámetros por defecto:

```
./target/release/jsp -f data/facil.txt simulated-annealing
./target/release/jsp -f data/medio.txt simulated-annealing
./target/release/jsp -f data/dificil.txt simulated-annealing
```

Para ejecutar la experimentación completa de diez repeticiones por instancia:

```
./target/release/benchmark
```

La ayuda de la interfaz de línea de comandos se obtiene con:

```
./target/release/jsp --help
./target/release/jsp simulated-annealing --help
```

A.2. Estructura del repositorio

```
jsp/
|-- Cargo.toml
|-- README.txt
|-- data/
|   |-- facil.txt
|   |-- medio.txt
|   `-- dificil.txt
|-- src/
|   |-- domain/
|   |-- solver/
|   |-- ui/
|   `-- main.rs
```

```
|-- benches/
|   `-- benchmark.rs
`-- docs/
    `-- informe/
```

El archivo `README.txt` contiene las instrucciones de ejecución y la identificación de todos los integrantes del grupo, conforme a lo solicitado en el enunciado de la tarea.

A.3. Instancias utilizadas

Las tres instancias se almacenan en la carpeta `data` y utilizan el formato estándar del JSP: la primera línea contiene el número de trabajos y de máquinas; cada línea siguiente contiene pares (máquina, duración) en el orden tecnológico del trabajo correspondiente.

Tabla A.1: Instancias utilizadas en la experimentación.

Instancia	Trabajos	Máquinas	Operaciones
Fácil	6	6	36
Medio	15	10	150
Difícil	50	10	500